

test_linked_list.cpp

Este documento contiene la implementación completa del módulo de pruebas requerido para la práctica del ADT Linked List, integrando todos los axiomas, casos límite y pruebas de propiedades.



```
#include <iostream>
#include <cassert>
#include <string>
#include "LinkedList.hpp" // Cambiar a .h si tu archivo se llama así

using namespace std;

// --- Infraestructura minima de reporte de pruebas ---
static int testsRun = 0;
static int testsPassed = 0;

void check(const string& testName, bool condition) {
    testsRun++;
    if (condition) {
        testsPassed++;
        cout << "[OK] " << testName << endl;
    } else {
        cout << "[FALLO] " << testName << endl;
    }
}

// Axioma 1: lista inicialmente vacia
void testAxioma1_ListaVacía() {
```

```

LinkedList<int> L;
check("Axioma 1: isEmpty() en lista nueva", L.isEmpty() == true);
check("Axioma 1: size() en lista nueva", L.size() == 0);
}

// Axioma 2: insercion incrementa el tamaño
void testAxioma2_InsercionIncrementaTamano() {
    LinkedList<int> L;
    int oldSize = L.size();
    L.insertFront(10);
    check("Axioma 2: size(insertFront(L,e)) == size(L)+1", L.size() == oldSize
+ 1);
}

// Axioma 3: el elemento insertado es el primero
void testAxioma3_PrimerTrasInsertar() {
    LinkedList<int> L;
    L.insertFront(10);
    L.insertFront(20);
    check("Axioma 3: first(insertFront(L,e)) == e", L.first() == 20);
}

// Axioma 4: first(L) == get(L,0)
void testAxioma4_PrimerEsPosicionCero() {
    LinkedList<int> L;
    L.insertFront(10);
    L.insertFront(20);
    check("Axioma 4: first(L) == get(L,0)", L.first() == L.get(0));
}

// Axioma 5: la eliminacion disminuye el tamaño
void testAxioma5_EliminacionDisminuyeTamano() {
    LinkedList<int> L;
    L.insertFront(10);

```

```
L.insertFront(20);
int oldSize = L.size();
L.removeFront();
check("Axioma 5: size(removeFront(L)) = size(L)-1", L.size() == oldSize
- 1);
}
```

// Axioma 6: eliminar el frente deja como primero al antiguo segundo elemento

```
void testAxioma6_SegundoPasaAPrimero() {
    LinkedList<int> L;
    L.insertFront(30);
    L.insertFront(20);
    L.insertFront(10); // L = [10, 20, 30]
    L.removeFront(); // L = [20, 30]
    check("Axioma 6: tras removeFront, first(L) es el antiguo segundo",
L.first() == 20);
}
```

// Axioma 7: contains

```
void testAxioma7_Contains() {
    LinkedList<int> L;
    L.insertFront(30);
    L.insertFront(20);
    L.insertFront(10);
    check("Axioma 7: contains(L,20) == true", L.contains(20) == true);
    check("Axioma 7: contains(L,100) == false", L.contains(100) == false);
}
```

// Axioma 8: indexOf

```
void testAxioma8_IndexOf() {
    LinkedList<int> L;
    L.insertFront(30);
    L.insertFront(20);
```

```

L.insertFront(10);
check("Axioma 8: indexOf(L,20) = 1", L.indexOf(20) == 1);
check("Axioma 8: indexOf(L,100) = -1 (convencion)", L.indexOf(100)
== -1);
}

```

// Pruebas basadas en propiedades (Seccion 12)

```

void testPropiedadInsertFront() {
    int valores[] = {5, -3, 0, 1000, 42};
    int n = 5;
    for (int i = 0; i < n; i++) {
        LinkedList<int> L;
        L.insertFront(1);
        L.insertFront(2);
        int oldSize = L.size();
        int e = valores[i];
        L.insertFront(e);
        check("Propiedad: first(L) = e para e=" + to_string(e), L.first() ==
e);
        check("Propiedad: size(L) = size(L_anterior)+1 para e=" +
to_string(e), L.size() == oldSize + 1);
    }
}

```

// Casos limite (Seccion 13)

```

void testCasosLimite() {
    // 1. Lista vacia
    LinkedList<int> vacia;
    check("Caso limite: lista vacia → isEmpty() = true", vacia.isEmpty() ==
true);

```

// 2. Lista con un elemento

```

LinkedList<int> unElemento;
unElemento.insertFront(7);

```

```
check("Caso limite: un elemento → size() = 1", unElemento.size() == 1);  
check("Caso limite: un elemento → first() = 7", unElemento.first() == 7);
```

```
// 3. Lista con dos elementos  
LinkedList<int> dosElementos;  
dosElementos.insertFront(2);  
dosElementos.insertFront(1);  
check("Caso limite: dos elementos → get(0) = 1",  
dosElementos.get(0) == 1);  
check("Caso limite: dos elementos → get(1) = 2",  
dosElementos.get(1) == 2);
```

```
// 4. Insercion repetida por el frente  
LinkedList<int> repetida;  
for (int i = 1; i ≤ 5; i++) {  
    repetida.insertFront(i);  
}  
check("Caso limite: 5 inserciones → size() = 5", repetida.size() == 5);  
check("Caso limite: 5 inserciones → first() = 5", repetida.first() == 5);
```

```
// 5. Eliminacion hasta dejar la lista vacia  
LinkedList<int> vaciar;  
vaciar.insertFront(1);  
vaciar.insertFront(2);  
vaciar.insertFront(3);  
while (!vaciar.isEmpty()) {  
    vaciar.removeFront();  
}  
check("Caso limite: remover todos los elementos → isEmpty() = true",  
vaciar.isEmpty() == true);  
check("Caso limite: remover todos los elementos → size() = 0",  
vaciar.size() == 0);
```

```
// 6. Búsqueda de un elemento inexistente
LinkedList<int> busqueda;
busqueda.insertFront(1);
busqueda.insertFront(2);
check("Caso limite: elemento inexistente → contains() = false",
busqueda.contains(999) = false);
check("Caso limite: elemento inexistente → indexOf() = -1",
busqueda.indexOf(999) = -1);
```

```
// 7. Búsqueda de elementos repetidos
LinkedList<int> repetidos;
repetidos.insertFront(5);
repetidos.insertFront(5);
repetidos.insertFront(5);
check("Caso limite: elementos repetidos → contains() = true",
repetidos.contains(5) = true);
check("Caso limite: elementos repetidos → indexOf() apunta al
primero", repetidos.indexOf(5) = 0);
```

```
// 8. clear() sobre lista vacia
LinkedList<int> clearVacia;
clearVacia.clear();
check("Caso limite: clear() sobre lista vacia → isEmpty() = true",
clearVacia.isEmpty() = true);
```

```
// 9. clear() sobre lista con elementos
LinkedList<int> clearConElementos;
clearConElementos.insertFront(1);
clearConElementos.insertFront(2);
clearConElementos.insertFront(3);
clearConElementos.clear();
check("Caso limite: clear() con elementos → isEmpty() = true",
clearConElementos.isEmpty() = true);
```

```

    check("Caso limite: clear() con elementos → size() = 0",
clearConElementos.size() == 0);
}

// Secuencia de prueba de la Sección 14
void testSecuenciaSeccion14() {
    LinkedList<int> L;
    L.insertFront(10);
    L.insertFront(20);
    L.insertFront(30); // L = [30, 20, 10]

    check("Secuencia: get(0) = 30", L.get(0) == 30);
    check("Secuencia: get(1) = 20", L.get(1) == 20);
    check("Secuencia: get(2) = 10", L.get(2) == 10);

    L.removeFront(); // L = [20, 10]

    check("Secuencia: tras removeFront, get(0) = 20", L.get(0) == 20);
    check("Secuencia: tras removeFront, get(1) = 10", L.get(1) == 10);
    check("Secuencia: tras removeFront, size() = 2", L.size() == 2);
}

// Orquestador principal
void testLinkedList() {
    testAxioma1_ListaVacía();
    testAxioma2_InsercionIncrementaTamano();
    testAxioma3_PrimerTrasInsertar();
    testAxioma4_PrimerEsPosicionCero();
    testAxioma5_EliminacionDisminuyeTamano();
    testAxioma6_SegundoPasaAPrimero();
    testAxioma7_Contains();
    testAxioma8_IndexOf();
    testPropiedadInsertFront();
    testCasosLimite();
}

```

```
testSecuenciaSeccion14();  
}  
  
int main() {  
    testLinkedList();  
    cout << endl;  
    cout << "Resumen: " << testsPassed << "/" << testsRun << " pruebas  
aprobadas." << endl;  
    return (testsPassed == testsRun) ? 0 : 1;  
}
```